

December 11, 2025

```
[122]: # Load data
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
df = pd.read_csv("../data/train_sample.csv")

[123]: # Check the dataset
print(df.info())
print(df.describe())
# Check the data of each column
print("\nData sample:\n", df.head())
print("\nMissing values per column:\n", df.isnull().sum().
      ↪sort_values(ascending=False).head(10))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 28 columns):
```

#	Column	Non-Null Count	Dtype
0	ID	1000 non-null	object
1	Customer_ID	1000 non-null	object
2	Month	1000 non-null	object
3	Name	902 non-null	object
4	Age	1000 non-null	object
5	SSN	1000 non-null	object
6	Occupation	1000 non-null	object
7	Annual_Income	1000 non-null	object
8	Monthly_Inhand_Salary	845 non-null	float64
9	Num_Bank_Accounts	1000 non-null	int64
10	Num_Credit_Card	1000 non-null	int64
11	Interest_Rate	1000 non-null	int64
12	Num_of_Loan	1000 non-null	object
13	Type_of_Loan	832 non-null	object
14	Delay_from_due_date	1000 non-null	int64
15	Num_of_Delayed_Payment	924 non-null	object
16	Changed_Credit_Limit	1000 non-null	object
17	Num_Credit_Inquiries	975 non-null	float64

```

18 Credit_Mix          1000 non-null object
19 Outstanding_Debt    1000 non-null object
20 Credit_Utilization_Ratio 1000 non-null float64
21 Credit_History_Age  905 non-null object
22 Payment_of_Min_Amount 1000 non-null object
23 Total_EMI_per_month 1000 non-null float64
24 Amount_invested_monthly 954 non-null object
25 Payment_Behaviour    1000 non-null object
26 Monthly_Balance      983 non-null float64
27 Credit_Score         1000 non-null object

```

dtypes: float64(5), int64(4), object(19)

memory usage: 218.9+ KB

None

	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card \
count	845.000000	1000.000000	1000.000000
mean	4462.535093	10.935000	26.783000
std	3123.601366	79.197911	147.942668
min	519.128750	0.000000	1.000000
25%	1828.240000	3.000000	4.000000
50%	3260.465000	6.000000	5.000000
75%	6300.581667	8.000000	7.000000
max	12432.610000	1488.000000	1385.000000

	Interest_Rate	Delay_from_due_date	Num_Credit_Inquiries \
count	1000.000000	1000.000000	975.000000
mean	83.111000	21.693000	13.694359
std	510.892781	16.364857	104.269656
min	1.000000	-2.000000	0.000000
25%	7.000000	9.000000	2.000000
50%	14.000000	17.000000	4.000000
75%	20.000000	28.000000	9.000000
max	5565.000000	67.000000	1936.000000

	Credit_Utilization_Ratio	Total_EMI_per_month	Monthly_Balance
count	1000.000000	1000.000000	983.000000
mean	32.170609	1472.216076	409.801781
std	5.142146	7972.497257	217.698963
min	21.744884	0.000000	7.129357
25%	27.979514	29.045533	270.231241
50%	32.163236	69.685459	348.904666
75%	36.486123	185.631542	474.345098
max	46.194238	81441.000000	1184.899694

Data sample:

	ID	Customer_ID	Month	Name	Age	SSN	Occupation \
0	0x1602	CUS_0xd40	January	Aaron Maashoh	23	821-00-0265	Scientist
1	0x1603	CUS_0xd40	February	Aaron Maashoh	23	821-00-0265	Scientist
2	0x1604	CUS_0xd40	March	Aaron Maashoh	-500	821-00-0265	Scientist

3	0x1605	CUS_0xd40	April	Aaron Maashoh	23	821-00-0265	Scientist
4	0x1606	CUS_0xd40	May	Aaron Maashoh	23	821-00-0265	Scientist

	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	...	Credit_Mix	\
0	19114.12	1824.843333	3	...	-	
1	19114.12	NaN	3	...	Good	
2	19114.12	NaN	3	...	Good	
3	19114.12	NaN	3	...	Good	
4	19114.12	1824.843333	3	...	Good	

	Outstanding_Debt	Credit_Utilization_Ratio	Credit_History_Age	\
0	809.98	26.822620	22 Years and 1 Months	
1	809.98	31.944960	NaN	
2	809.98	28.609352	22 Years and 3 Months	
3	809.98	31.377862	22 Years and 4 Months	
4	809.98	24.797347	22 Years and 5 Months	

	Payment_of_Min_Amount	Total_EMI_per_month	Amount_invested_monthly	\
0	No	49.574949	80.41529543900253	
1	No	49.574949	118.28022162236736	
2	No	49.574949	81.699521264648	
3	No	49.574949	199.4580743910713	
4	No	49.574949	41.420153086217326	

	Payment_Behaviour	Monthly_Balance	Credit_Score
0	High_spent_Small_value_payments	312.494089	Good
1	Low_spent_Large_value_payments	284.629162	Good
2	Low_spent_Medium_value_payments	331.209863	Good
3	Low_spent_Small_value_payments	223.451310	Good
4	High_spent_Medium_value_payments	341.489231	Good

[5 rows x 28 columns]

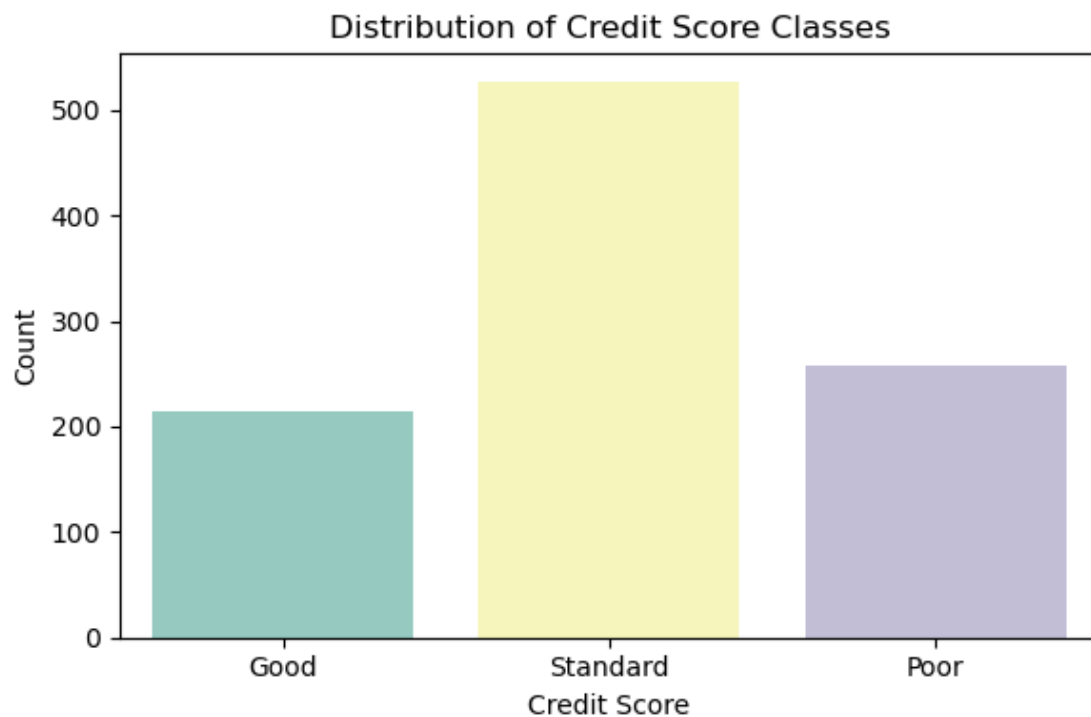
Missing values per column:

Type_of_Loan	168
Monthly_Inhand_Salary	155
Name	98
Credit_History_Age	95
Num_of_Delayed_Payment	76
Amount_invested_monthly	46
Num_Credit_Inquiries	25
Monthly_Balance	17
ID	0
Changed_Credit_Limit	0

dtype: int64

```
[124]: # EDA Visualize distributions of key features

# Distribution of target variable
#1 Target variable distribution
plt.figure(figsize=(6,4))
sns.countplot(data=df, x="Credit_Score", hue="Credit_Score", palette="Set3")
plt.title("Distribution of Credit Score Classes")
plt.xlabel("Credit Score")
plt.ylabel("Count")
plt.tight_layout()
plt.show()# Explore feature distributions by Credit Score
```



```
[125]: #Distributions of key features
# 1 Annual Income distribution
df_plot = df.copy()

df_plot["Annual_Income"] = (
    df_plot["Annual_Income"].astype(str)
    .str.replace(",", "", regex=False)
    .str.replace("_", "", regex=False)
)
```

```

df_plot["Annual_Income"] = pd.to_numeric(df_plot["Annual_Income"],
    ↪errors="coerce")

cap_value = df_plot["Annual_Income"].quantile(0.99)
df_plot["Annual_Income"] = np.minimum(df_plot["Annual_Income"], cap_value)

plt.figure(figsize=(6,4))
sns.histplot(
    data=df_plot,
    x="Annual_Income",
    kde=True,
    bins=30,
    color="skyblue"
)
plt.title("Distribution of Annual Income (Capped)")
plt.xlabel("Annual Income (USD)")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 2 Credit Utilization Ratio distribution
df_plot = df.copy()
df_plot["Credit_Utilization_Ratio"] = pd.
    ↪to_numeric(df_plot["Credit_Utilization_Ratio"], errors="coerce")

cap = df_plot["Credit_Utilization_Ratio"]
df_plot["Credit_Utilization_Ratio"] = np.
    ↪minimum(df_plot["Credit_Utilization_Ratio"], cap)

plt.figure(figsize=(6,4))
sns.histplot(
    data=df_plot,
    x="Credit_Utilization_Ratio",
    kde=True,
    bins=30,
    color="skyblue"
)
plt.title("Distribution of Credit Utilization Ratio (Capped)")
plt.xlabel("Credit Utilization Ratio")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# 3 Payment of Minimum Amount distribution
df_eda = df[df["Payment_of_Min_Amount"].isin(["Yes", "No"])]

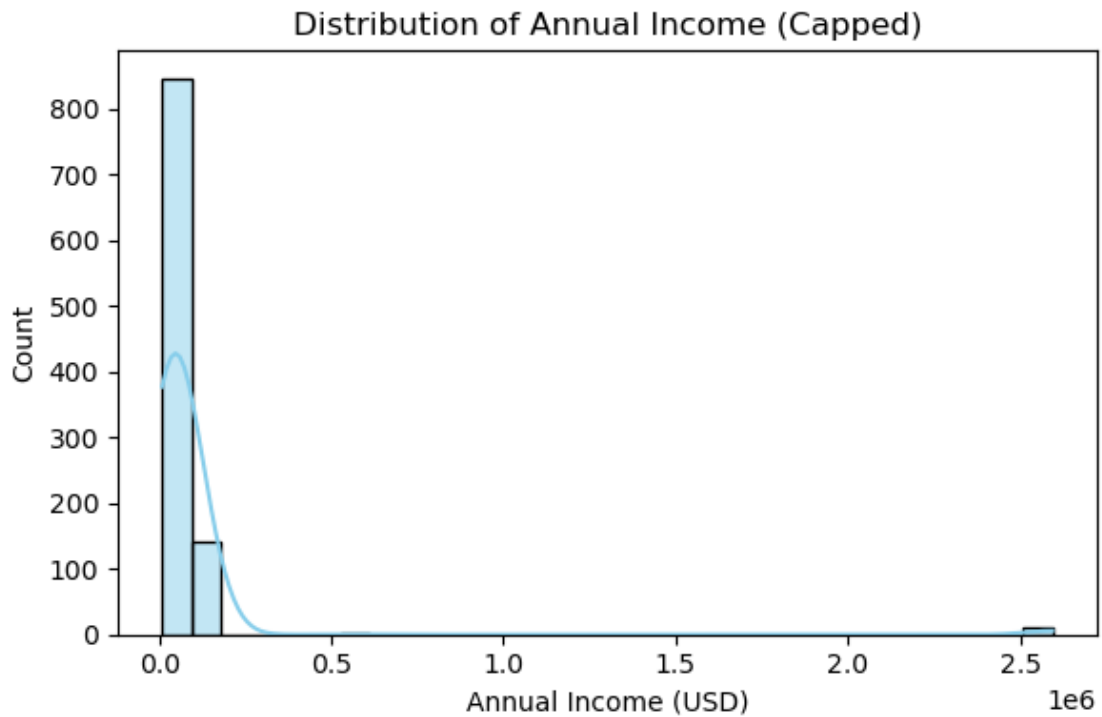
plt.figure(figsize=(6,4))

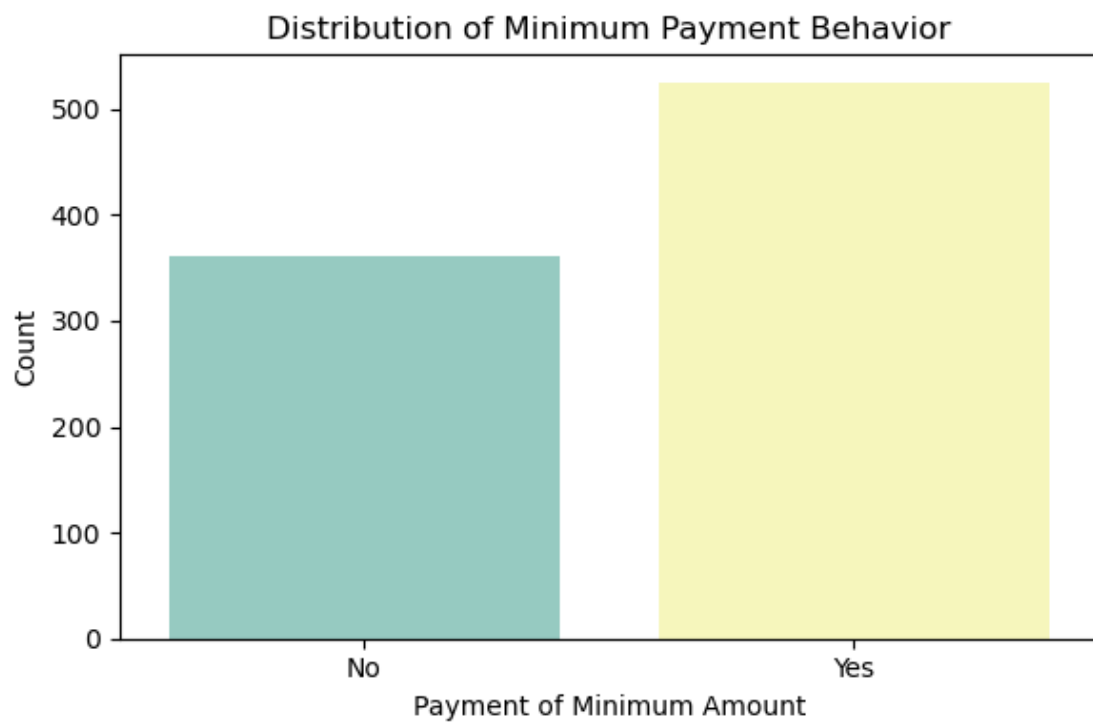
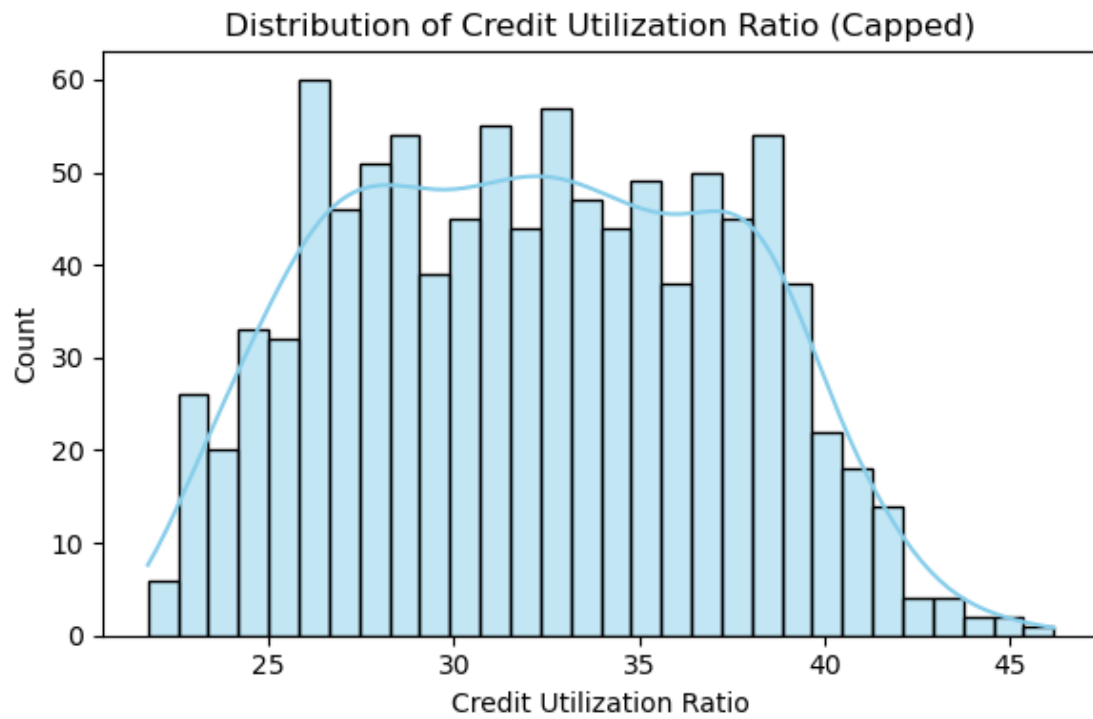
```

```

sns.countplot(data=df_eda, x="Payment_of_Min_Amount",
              hue="Payment_of_Min_Amount", palette="Set3")
plt.title("Distribution of Minimum Payment Behavior")
plt.xlabel("Payment of Minimum Amount")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

```





```

[126]: # Feature distributions by Credit Score
# 1 Age distribution by Credit Score
df_plot = df.copy()

df_plot["Age"] = pd.to_numeric(df_plot["Age"], errors="coerce")

df_plot = df_plot[(df_plot["Age"] > 0) & (df_plot["Age"] < 100)]

plt.figure(figsize=(6,4))
sns.boxplot(
    data=df_plot,
    x="Credit_Score",
    hue="Credit_Score",
    y="Age",
    palette="Set3",
    order=["Good", "Standard", "Poor"]
)
plt.title("Age Distribution by Credit Score")
plt.xlabel("Credit Score")
plt.ylabel("Age")
plt.tight_layout()
plt.show()

# 2 Number of Bank Accounts distribution by Credit Score
df_plot = df.copy()
df_plot["Num_Bank_Accounts"] = pd.to_numeric(df_plot["Num_Bank_Accounts"],
errors="coerce")
df_plot = df_plot[df_plot["Num_Bank_Accounts"] >= 0]

plt.figure(figsize=(6,4))
sns.boxplot(
    data=df_plot,
    x="Credit_Score",
    hue="Credit_Score",
    y="Num_Bank_Accounts",
    order=["Good", "Standard", "Poor"],
    palette="Set3",
    width=0.5
)

plt.title("Number of Bank Accounts by Credit Score (Zoomed In)", fontsize=15,
weight='bold')
plt.xlabel("Credit Score Category", fontsize=12)
plt.ylabel("Number of Bank Accounts", fontsize=12)

plt.ylim(-1, 20)

```

```

sns.despine()
plt.tight_layout()
plt.show()

# 3 Outstanding Debt distribution by Credit Score
df_plot = df.copy()

df_plot["Outstanding_Debt"] = (
    df_plot["Outstanding_Debt"]
    .astype(str)
    .str.replace(",", "", regex=False)
    .replace("NA", np.nan)
)

df_plot["Outstanding_Debt"] = pd.to_numeric(df_plot["Outstanding_Debt"],
    ↪errors="coerce")

plt.figure(figsize=(6,4))
sns.violinplot(
    data=df_plot,
    y="Credit_Score",
    x="Outstanding_Debt",
    hue="Credit_Score",
    order=["Good", "Standard", "Poor"],
    palette="Set3",
    inner="quartile"
)

plt.title("Outstanding Debt by Credit Score", fontsize=15, weight='bold')
plt.xlabel("Outstanding Debt (dollars)", fontsize=12)
plt.ylabel("Credit Score Category", fontsize=12)

sns.despine()
plt.tight_layout()
plt.show()

# 4 Number of Delayed Payment distribution by Credit Score
df_plot = df.copy()
df_plot["Num_of_Delayed_Payment"] = pd.
    ↪to_numeric(df_plot["Num_of_Delayed_Payment"], errors="coerce")

cap = df_plot["Num_of_Delayed_Payment"].quantile(0.98)
df_plot["Num_of_Delayed_Payment"] = np.
    ↪minimum(df_plot["Num_of_Delayed_Payment"], cap)

```

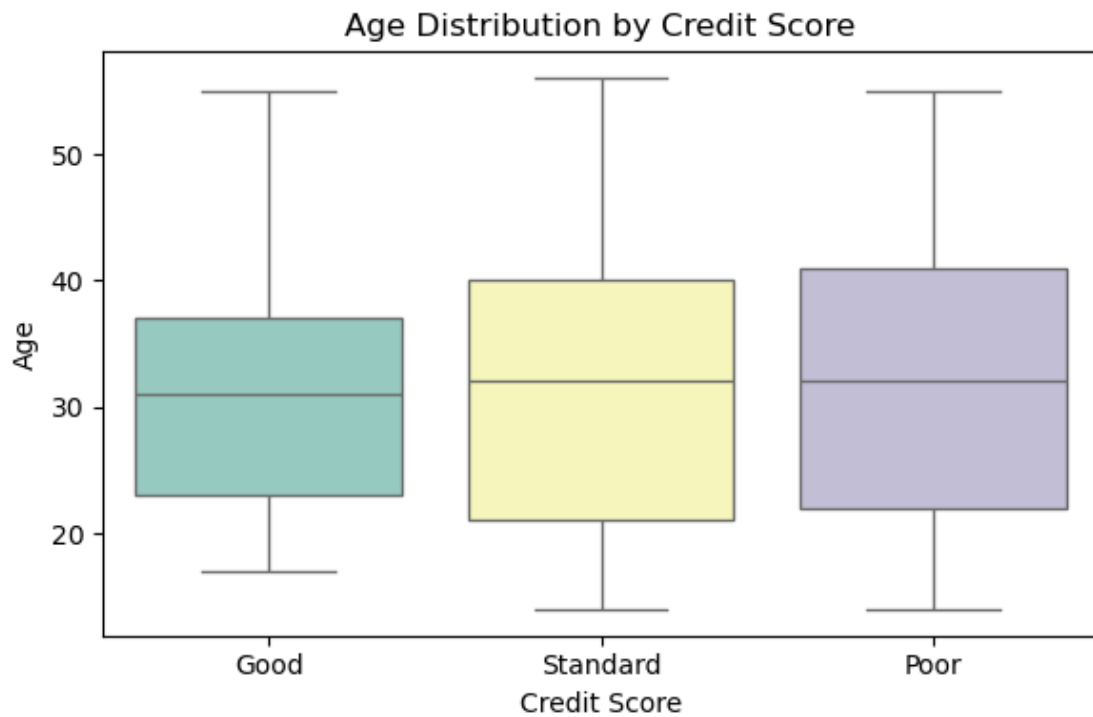
```

plt.figure(figsize=(6,4))
sns.boxplot(
    data=df_plot,
    x="Credit_Score",
    hue="Credit_Score",
    y="Num_of_Delayed_Payment",
    palette="Set3",
    order=["Good", "Standard", "Poor"]
)
plt.title("Delayed Payments by Credit Score (Capped at 98th Percentile)")
plt.xlabel("Credit Score")
plt.ylabel("Number of Delayed Payments")
plt.tight_layout()
plt.show()

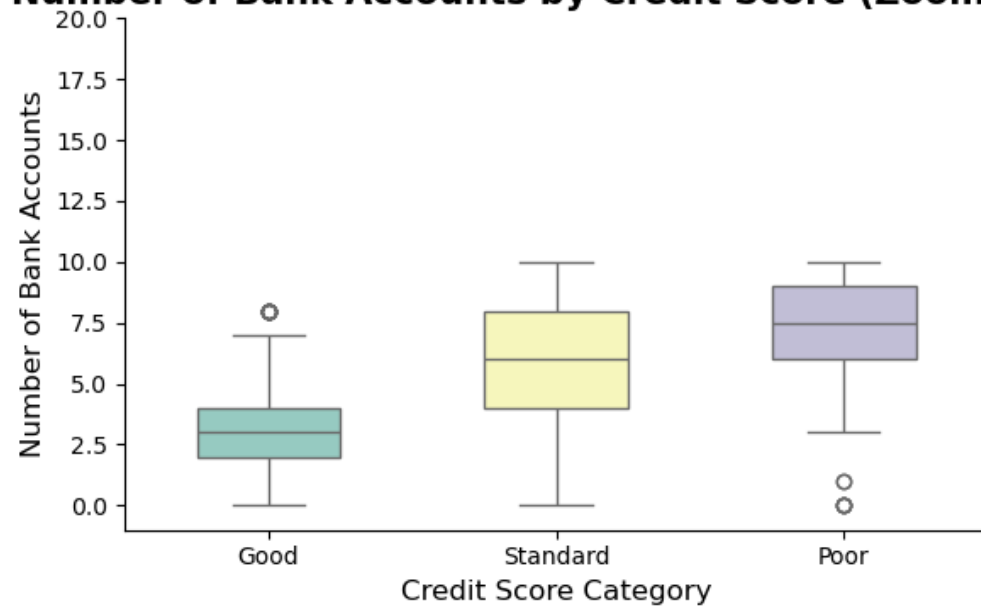
# 5 Payment Behaviour distribution by Credit Score
df_plot = df[df["Payment_Behaviour"] != "!@9#%8"].copy()

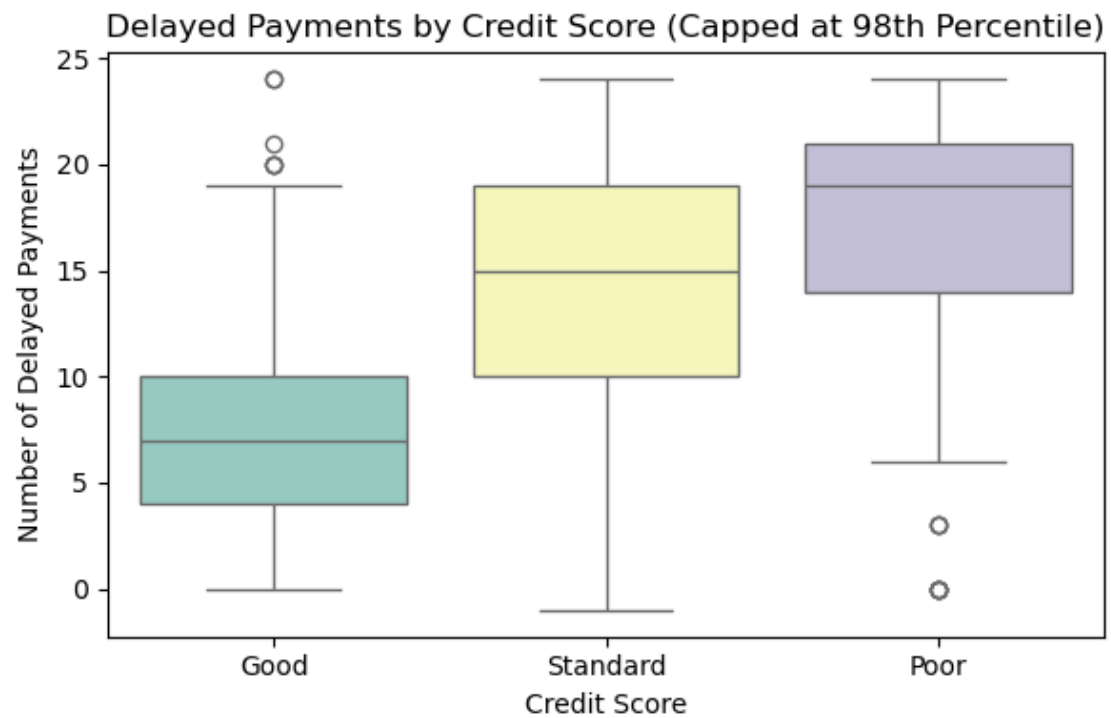
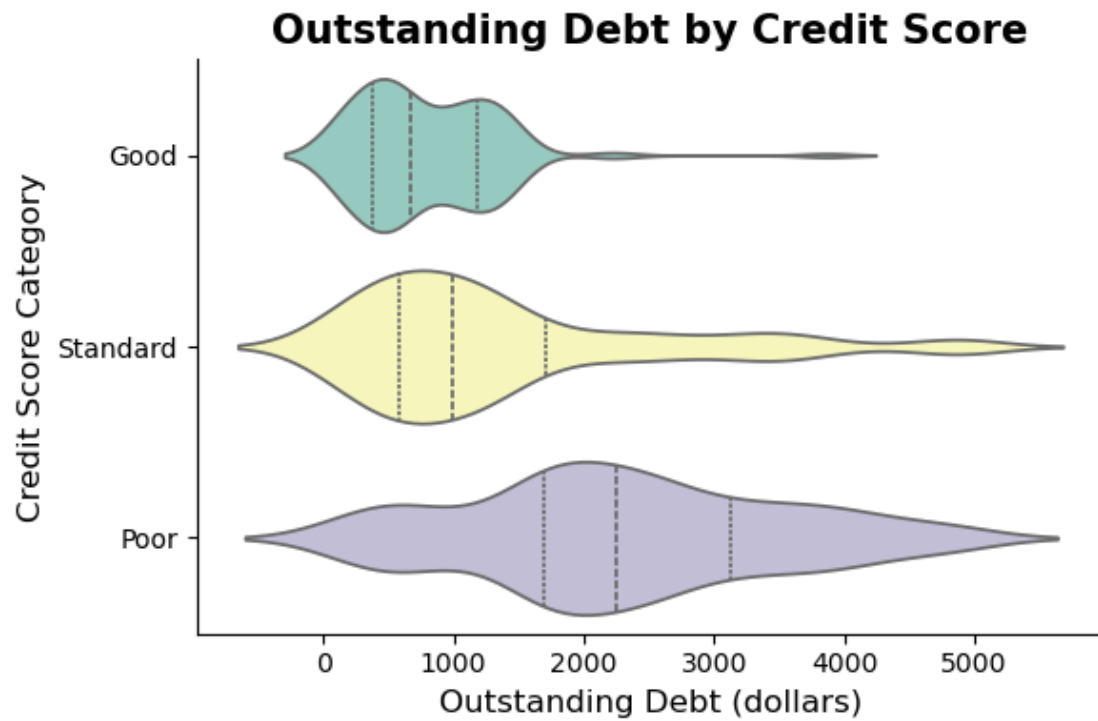
plt.figure(figsize=(10,4))
sns.countplot(
    data=df_plot,
    x="Payment_Behaviour",
    hue="Credit_Score",
    palette="Set3"
)
plt.xticks(rotation=45, ha="right")
plt.title("Payment Behaviour by Credit Score")
plt.tight_layout()
plt.show()

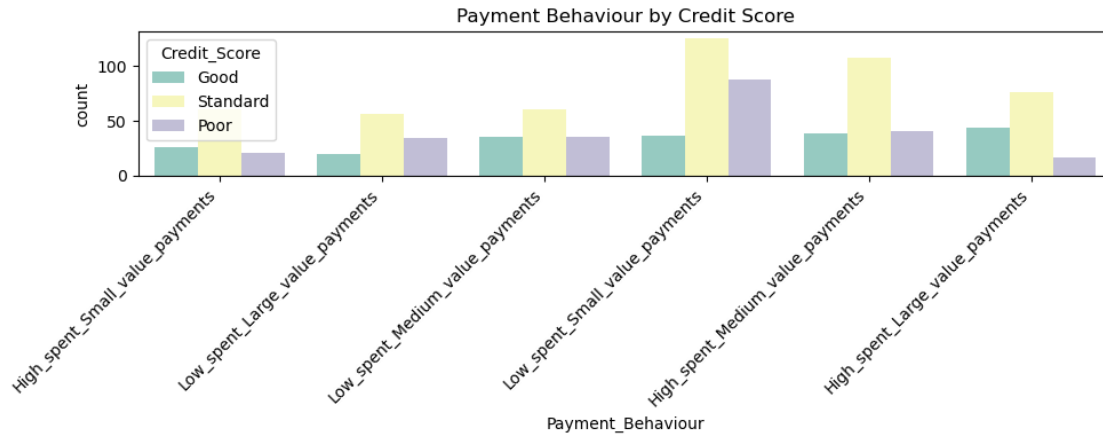
```



Number of Bank Accounts by Credit Score (Zoomed In)







```
[127]: # Correlation heatmap

# Create cleaned numeric copy
df_corr = df.copy()

# Define candidate numeric columns (from your list)
numeric_cols = [
    "Age",
    "Annual_Income",
    "Monthly_Inhand_Salary",
    "Num_Bank_Accounts",
    "Num_Credit_Card",
    "Interest_Rate",
    "Num_of_Loan",
    "Delay_from_due_date",
    "Num_of_Delayed_Payment",
    "Num_Credit_Inquiries",
    "Outstanding_Debt",
    "Credit_Utilization_Ratio",
    "Total_EMI_per_month",
    "Amount_invested_monthly",
    "Monthly_Balance"
]

# Fix dtype issues
for col in numeric_cols:
    df_corr[col] = (
        df_corr[col]
        .astype(str)
        .str.replace(",", "", regex=False)
```

```

        .str.replace("_", "", regex=False)
    )
    df_corr[col] = pd.to_numeric(df_corr[col], errors="coerce")

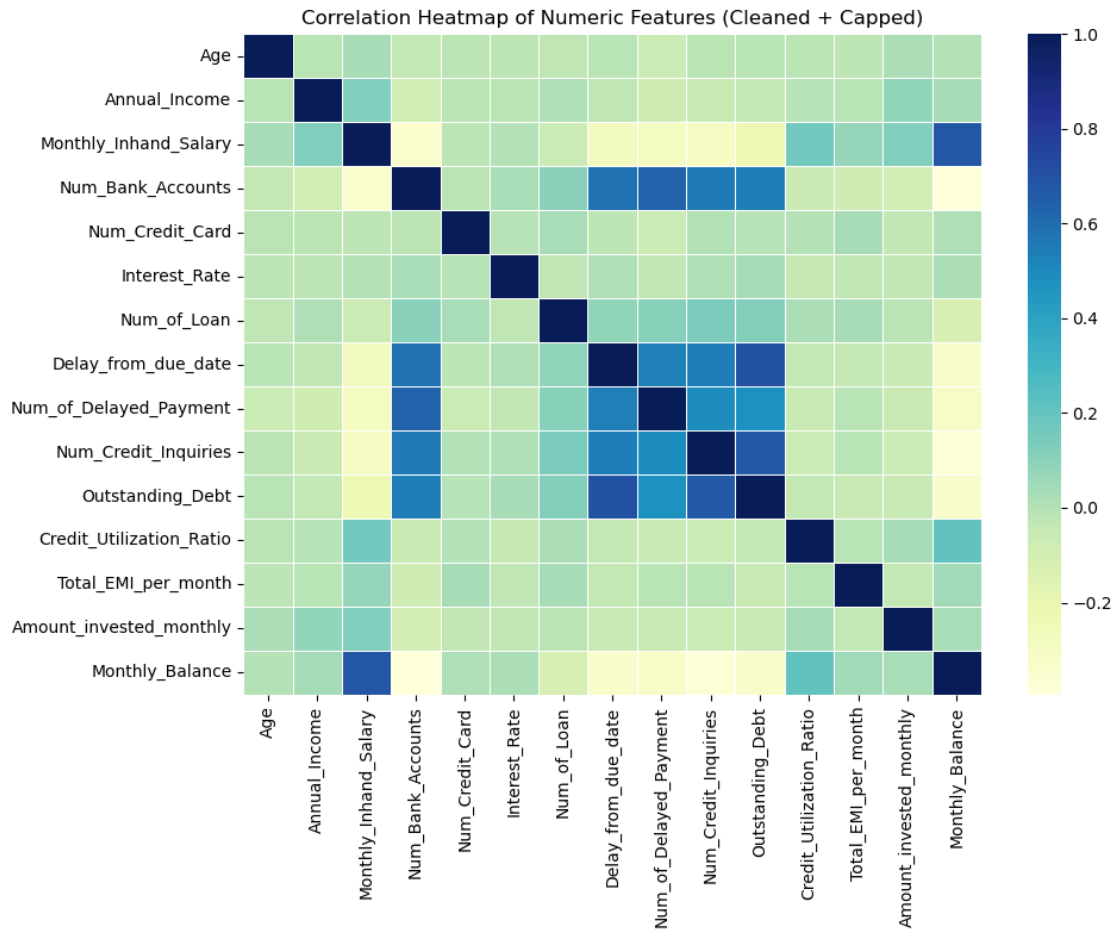
# Handle invalid values
df_corr.loc[df_corr["Age"] < 0, "Age"] = np.nan
df_corr.loc[df_corr["Delay_from_due_date"] < 0, "Delay_from_due_date"] = np.nan

# Cap outliers at 99th percentile
for col in numeric_cols:
    if df_corr[col].dtype in [np.float64, np.int64]:
        cap = df_corr[col].quantile(0.99)
        df_corr[col] = np.minimum(df_corr[col], cap)

# Compute correlation matrix
corr_matrix = df_corr[numeric_cols].corr()

# Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(
    corr_matrix,
    cmap="YlGnBu",
    annot=False,
    linewidths=0.5
)
plt.title("Correlation Heatmap of Numeric Features (Cleaned + Capped)")
plt.tight_layout()
plt.show()

```



```
[128]: # Data Cleaning
df_clean = df.copy()

def clean_numeric(s):
    return (
        s.astype(str)
        .str.replace(",", "", regex=False)
        .str.replace("_", "", regex=False)
        .replace(["NA", "NM", "None", ""], np.nan)
    )

numeric_object_cols = [
    "Age",
    "Annual_Income",
    "Num_of_Loan",
    "Num_of_Delayed_Payment",
    "Changed_Credit_Limit",
    "Outstanding_Debt",
```

```

    "Amount_invested_monthly",
]

for col in numeric_object_cols:
    df_clean[col] = clean_numeric(df_clean[col])
    df_clean[col] = pd.to_numeric(df_clean[col], errors="coerce")

# Age impossible (<0 or >100)
df_clean = df_clean[(df_clean["Age"] > 0) & (df_clean["Age"] < 100)]

# Num_of_Loan cannot be negative
df_clean = df_clean[df_clean["Num_of_Loan"] >= 0]

# Num_of_Delayed_Payment cannot be negative
df_clean = df_clean[(df_clean["Num_of_Delayed_Payment"] >= 0)]

# Outstanding_Debt cannot be negative
df_clean = df_clean[(df_clean["Outstanding_Debt"] >= 0)]

# Credit_Utilization_Ratio cannot be negative or above 100
# (if >100 it's fraud but not impossible, so only drop <0 here)
df_clean = df_clean[df_clean["Credit_Utilization_Ratio"] >= 0]

# Delay_from_due_date cannot be negative
df_clean = df_clean[df_clean["Delay_from_due_date"] >= 0]

# Interest rate cannot exceed 100% in any real consumer loan scenario
df_clean = df_clean[df_clean["Interest_Rate"] <= 100]

# Monthly_Balance: remove thousand separators and cast to numeric
df_clean["Monthly_Balance"] = (
    df_clean["Monthly_Balance"]
    .astype(str)
    .str.replace(",", "")
)
df_clean["Monthly_Balance"] = pd.to_numeric(df_clean["Monthly_Balance"],
errors="coerce")

# Parse Credit_History_Age: "XX Years and YY Months" -> total months
def parse_credit_age(x):
    if pd.isna(x):
        return np.nan
    parts = x.split(" ")
    years = int(parts[0])
    months = int(parts[3])
    return years * 12 + months

```

```
df_clean["Credit_History_Age"] = df_clean["Credit_History_Age"].
    ↪ apply(parse_credit_age)

print("Shape before cleaning:", df.shape)
print("Shape after cleaning :", df_clean.shape)
print(df_clean.info())
print(df_clean.head())
print(df_clean.describe())
```

Shape before cleaning: (1000, 28)

Shape after cleaning : (834, 28)

<class 'pandas.core.frame.DataFrame'>

Index: 834 entries, 0 to 999

Data columns (total 28 columns):

#	Column	Non-Null Count	Dtype
0	ID	834 non-null	object
1	Customer_ID	834 non-null	object
2	Month	834 non-null	object
3	Name	754 non-null	object
4	Age	834 non-null	int64
5	SSN	834 non-null	object
6	Occupation	834 non-null	object
7	Annual_Income	834 non-null	float64
8	Monthly_Inhand_Salary	709 non-null	float64
9	Num_Bank_Accounts	834 non-null	int64
10	Num_Credit_Card	834 non-null	int64
11	Interest_Rate	834 non-null	int64
12	Num_of_Loan	834 non-null	int64
13	Type_of_Loan	705 non-null	object
14	Delay_from_due_date	834 non-null	int64
15	Num_of_Delayed_Payment	834 non-null	float64
16	Changed_Credit_Limit	817 non-null	float64
17	Num_Credit_Inquiries	811 non-null	float64
18	Credit_Mix	834 non-null	object
19	Outstanding_Debt	834 non-null	float64
20	Credit_Utilization_Ratio	834 non-null	float64
21	Credit_History_Age	749 non-null	float64
22	Payment_of_Min_Amount	834 non-null	object
23	Total_EMI_per_month	834 non-null	float64
24	Amount_invested_monthly	801 non-null	float64
25	Payment_Behaviour	834 non-null	object
26	Monthly_Balance	818 non-null	float64
27	Credit_Score	834 non-null	object

dtypes: float64(11), int64(6), object(11)

memory usage: 189.0+ KB

None

ID Customer_ID Month Name Age SSN Occupation \

0	0x1602	CUS_0xd40	January	Aaron Maashoh	23	821-00-0265	Scientist
3	0x1605	CUS_0xd40	April	Aaron Maashoh	23	821-00-0265	Scientist
5	0x1607	CUS_0xd40	June	Aaron Maashoh	23	821-00-0265	Scientist
6	0x1608	CUS_0xd40	July	Aaron Maashoh	23	821-00-0265	Scientist
7	0x1609	CUS_0xd40	August	NaN	23	#F%\$D@*&8	Scientist

	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	...	Credit_Mix	\
0	19114.12	1824.843333	3	...	-	
3	19114.12	NaN	3	...	Good	
5	19114.12	NaN	3	...	Good	
6	19114.12	1824.843333	3	...	Good	
7	19114.12	1824.843333	3	...	Good	

	Outstanding_Debt	Credit_Utilization_Ratio	Credit_History_Age	\
0	809.98	26.822620	265.0	
3	809.98	31.377862	268.0	
5	809.98	27.262259	270.0	
6	809.98	22.537593	271.0	
7	809.98	23.933795	NaN	

	Payment_of_Min_Amount	Total_EMI_per_month	Amount_invested_monthly	\
0	No	49.574949	80.415295	
3	No	49.574949	199.458074	
5	No	49.574949	62.430172	
6	No	49.574949	178.344067	
7	No	49.574949	24.785217	

	Payment_Behaviour	Monthly_Balance	Credit_Score
0	High_spent_Small_value_payments	312.494089	Good
3	Low_spent_Small_value_payments	223.451310	Good
5	!@9#%8	340.479212	Good
6	Low_spent_Small_value_payments	244.565317	Good
7	High_spent_Medium_value_payments	358.124168	Standard

[5 rows x 28 columns]

	Age	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	\
count	834.000000	8.340000e+02	709.000000	834.000000	
mean	31.730216	1.932651e+05	4405.233139	8.874101	
std	10.373013	1.465051e+06	3116.781613	55.708465	
min	14.000000	8.701545e+03	519.128750	0.000000	
25%	22.000000	2.018802e+04	1828.240000	4.000000	
50%	32.000000	4.134558e+04	3224.935000	6.000000	
75%	39.000000	7.392846e+04	6284.613333	8.000000	
max	56.000000	2.233578e+07	12432.610000	1414.000000	

	Num_Credit_Card	Interest_Rate	Num_of_Loan	Delay_from_due_date	\
count	834.000000	834.000000	834.000000	834.000000	
mean	26.197842	15.038369	6.500000	22.065947	

std	143.920064	9.116215	60.603314	16.133647
min	1.000000	1.000000	0.000000	0.000000
25%	4.000000	7.000000	2.000000	10.000000
50%	5.000000	14.000000	3.000000	17.000000
75%	7.000000	20.000000	6.000000	29.000000
max	1381.000000	34.000000	1464.000000	67.000000

	Num_of_Delayed_Payment	Changed_Credit_Limit	Num_Credit_Inquiries	\
count	834.000000	817.000000	811.000000	
mean	29.342926	10.586916	15.516646	
std	200.135590	6.683302	114.239463	
min	0.000000	-4.260000	0.000000	
25%	9.000000	5.620000	2.000000	
50%	15.000000	9.330000	5.000000	
75%	19.000000	15.060000	9.000000	
max	3318.000000	32.640000	1936.000000	

	Outstanding_Debt	Credit_Utilization_Ratio	Credit_History_Age	\
count	834.000000	834.000000	749.000000	
mean	1524.622290	32.219663	227.197597	
std	1230.686067	5.082285	105.442769	
min	35.540000	21.744884	4.000000	
25%	585.050000	28.061957	151.000000	
50%	1194.280000	32.191258	224.000000	
75%	2226.370000	36.472909	320.000000	
max	4984.820000	46.194238	404.000000	

	Total_EMI_per_month	Amount_invested_monthly	Monthly_Balance
count	834.000000	801.000000	818.000000
mean	1632.783198	584.452997	403.209765
std	8408.281256	1899.991933	212.640301
min	0.000000	13.066210	7.129357
25%	30.777012	80.484420	267.634259
50%	70.478333	147.573218	348.385581
75%	192.206483	274.456249	465.477338
max	81441.000000	10000.000000	1184.899694

```
[129]: # Check whether the dataset satisfies I.I.D. assumptions before dropping
        ↪non-predictive ID columns

# 1. Each Customer_ID should map to only one SSN
cid_ssn_conflict = df_clean.groupby("Customer_ID")["SSN"].transform("nunique")
        ↪> 1

# 2. Each Customer_ID should map to only one Name
cid_name_conflict = df_clean.groupby("Customer_ID")["Name"].
        ↪transform("nunique") > 1
```

```

# 3. Each SSN should map to only one Customer_ID
ssn_cid_conflict = df_clean.groupby("SSN")["Customer_ID"].transform("nunique")
    ↪> 1

# 4. Each SSN should map to only one Name
ssn_name_conflict = df_clean.groupby("SSN")["Name"].transform("nunique") > 1

# 5. ID should be unique, but check for safety
id_conflict = df_clean["ID"].duplicated(keep=False)

# Combine all conflict conditions
conflict_mask = (
    cid_ssn_conflict |
    cid_name_conflict |
    ssn_cid_conflict |
    ssn_name_conflict |
    id_conflict
)
# The misaligned rows (PS: NOT dropping them)
conflict_rows = df_clean[conflict_mask]
print("misaligned rows:", (conflict_rows.head(10)))

# Count unique valid person SSNs
persons = df_clean["SSN"].value_counts()
print("Number of unique persons:", len(persons))
print(persons.head())

"""Although each individual has 8 monthly records, this dataset is not treated
    ↪as a classical time series problem. In a true time series task, the target
    ↪at time t depends strictly on past values (e.g., ARIMA, forecasting). Here,
    ↪the goal is classification of credit score, using all available features at
    ↪each month. The temporal order does not define the predictive structure, and
    ↪the target is not modeled as a sequence. Therefore, the dataset is best
    ↪interpreted as panel data (repeated cross-sections) rather than a
    ↪forecasting problem."""

# Drop non-predictive non-grouping columns
df_clean = df_clean.drop(columns=["Customer_ID", "Name", "ID"])

"""According to entity-resolution literature (Christen 2012; Wooldridge 2010),
    ↪such inconsistencies in identity attributes are common in real-world and
    ↪synthetic datasets and should not be used as a criterion for dropping
    ↪observations, unless full entity resolution is performed.

```

Since the goal of this project is credit-score prediction-not identity,
 ↳reconciliation-these conflicts are treated as noisy metadata rather than,
 ↳erroneous samples. Dropping these rows would arbitrarily remove valid,
 ↳monthly observations and distort the distribution of the data.
 ""

misaligned rows:			ID	Customer_ID	Month	Name	Age
SSN	\						
0	0x1602	CUS_0xd40	January	Aaron Maashoh	23	821-00-0265	
3	0x1605	CUS_0xd40	April	Aaron Maashoh	23	821-00-0265	
5	0x1607	CUS_0xd40	June	Aaron Maashoh	23	821-00-0265	
6	0x1608	CUS_0xd40	July	Aaron Maashoh	23	821-00-0265	
7	0x1609	CUS_0xd40	August	NaN	23	#F%\$D@*&8	
24	0x1626	CUS_0xb891	January	Jasond	54	072-31-6145	
25	0x1627	CUS_0xb891	February	Jasond	54	072-31-6145	
26	0x1628	CUS_0xb891	March	Jasond	55	072-31-6145	
27	0x1629	CUS_0xb891	April	Jasond	55	072-31-6145	
28	0x162a	CUS_0xb891	May	Jasond	55	072-31-6145	

	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	\
0	Scientist	19114.12	1824.843333		3
3	Scientist	19114.12	NaN		3
5	Scientist	19114.12	NaN		3
6	Scientist	19114.12	1824.843333		3
7	Scientist	19114.12	1824.843333		3
24	Entrepreneur	30689.89	2612.490833		2
25	Entrepreneur	30689.89	2612.490833		2
26	Entrepreneur	30689.89	2612.490833		2
27	Entrepreneur	30689.89	2612.490833		2
28	Entrepreneur	30689.89	2612.490833		2

	...	Credit_Mix	Outstanding_Debt	Credit_Utilization_Ratio	\
0	...	-	809.98	26.822620	
3	...	Good	809.98	31.377862	
5	...	Good	809.98	27.262259	
6	...	Good	809.98	22.537593	
7	...	Good	809.98	23.933795	
24	...	Good	632.46	26.544229	
25	...	Good	632.46	35.279982	
26	...	Good	632.46	32.301163	
27	...	Good	632.46	38.132348	
28	...	Good	632.46	41.154317	

	Credit_History_Age	Payment_of_Min_Amount	Total_EMI_per_month	\
0	265.0	No	49.574949	
3	268.0	No	49.574949	
5	270.0	No	49.574949	
6	271.0	No	49.574949	

7	NaN	No	49.574949
24	207.0	No	16.415452
25	208.0	No	16.415452
26	209.0	NM	16.415452
27	210.0	No	16.415452
28	211.0	No	16.415452

	Amount_invested_monthly	Payment_Behaviour	Monthly_Balance \
0	80.415295	High_spent_Small_value_payments	312.494089
3	199.458074	Low_spent_Small_value_payments	223.451310
5	62.430172	!@9#%8	340.479212
6	178.344067	Low_spent_Small_value_payments	244.565317
7	24.785217	High_spent_Medium_value_payments	358.124168
24	81.228859	Low_spent_Large_value_payments	433.604773
25	124.881820	Low_spent_Small_value_payments	409.951812
26	83.406509	High_spent_Medium_value_payments	411.427123
27	272.334037	Low_spent_Small_value_payments	262.499594
28	10000.000000	Low_spent_Large_value_payments	359.374916

	Credit_Score
0	Good
3	Good
5	Good
6	Good
7	Standard
24	Standard
25	Standard
26	Standard
27	Standard
28	Standard

[10 rows x 28 columns]

Number of unique persons: 126

SSN

#F%\$D@*&8	47
352-67-4436	8
221-30-8554	8
751-65-6458	8
716-35-4628	8

Name: count, dtype: int64

[129]: 'According to entity-resolution literature (Christen 2012; Wooldridge 2010), such inconsistencies in identity attributes are common in real-world and synthetic datasets and should not be used as a criterion for dropping observations, unless full entity resolution is performed.\n\nSince the goal of this project is credit-score prediction-not identity reconciliation-these conflicts are treated as noisy metadata rather than erroneous samples. Dropping

these rows would arbitrarily remove valid monthly observations and distort the distribution of the data.\n'

```
[130]: # Check missing values for each column which are not written as NA
for col in df_clean.columns:
    uniques = df_clean[col].unique()
    print(f"\n--- {col} ---")
    print(f"Unique count: {len(uniques)}")
    # Only print first 20 to avoid flooding
    print(uniques[:20])
```

--- Month ---

Unique count: 8

['January' 'April' 'June' 'July' 'August' 'February' 'May' 'March']

--- Age ---

Unique count: 39

[23 28 34 54 55 21 31 33 30 24 44 40 41 35 36 39 37 20 45 46]

--- SSN ---

Unique count: 126

['821-00-0265' '#F%\$D@*&8' '004-07-5839' '486-85-3974' '072-31-6145'
'615-06-7821' '612-70-8987' '411-51-0676' '500-92-6408' '070-19-1622'
'366-68-1681' '221-30-8554' '342-90-2649' '414-53-2918' '328-33-6328'
'655-05-7666' '965-46-2491' '891-55-9364' '928-91-4452' '084-25-3745']

--- Occupation ---

Unique count: 16

['Scientist' '_____' 'Teacher' 'Engineer' 'Entrepreneur' 'Developer'
'Lawyer' 'Media_Manager' 'Doctor' 'Journalist' 'Manager' 'Accountant'
'Musician' 'Mechanic' 'Writer' 'Architect']

--- Annual_Income ---

Unique count: 136

[1.9114120e+04 3.4847840e+04 1.4316264e+05 3.0689890e+04 3.5547710e+04
7.3928460e+04 1.3131340e+05 1.0909427e+07 3.4081380e+04 1.1483841e+05
3.1370800e+04 3.3751270e+04 8.8640240e+04 5.4392160e+04 8.7015450e+03
2.5546260e+04 3.1993780e+04 9.2047080e+04 3.2284620e+04 9.7791420e+04]

--- Monthly_Inhand_Salary ---

Unique count: 134

[1824.84333333 nan 3037.98666667 12187.22
 2612.49083333 2853.30916667 5988.705 11242.78333333
 10469.20775939 2611.115 9843.8675 2825.23333333
 2948.60583333 7266.68666667 4766.68 519.12875
 2415.855 2942.14833333 7591.59 2898.385]

--- Num_Bank_Accounts ---

Unique count: 16

```
[ 3  2  1  7  4  0  8  5  6  9 10 1414 67 572
 91 528]
```

--- Num_Credit_Card ---

Unique count: 29

```
[ 4  5 1288  1  7  6 1029 488  8 1381 898  3 518 1005
 9 1189  2 10 514 943]
```

--- Interest_Rate ---

Unique count: 31

```
[ 3  6  8  4  5 15  7 12 20  1 14 32 16 17 10 31 25 18 19  9]
```

--- Num_of_Loan ---

Unique count: 12

```
[ 4  1  3 967  0  2  7  5  6  8  9 1464]
```

--- Type_of_Loan ---

Unique count: 98

```
['Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan'
 'Credit-Builder Loan' 'Auto Loan, Auto Loan, and Not Specified'
 'Not Specified' nan 'Credit-Builder Loan, and Mortgage Loan'
 'Not Specified, Auto Loan, and Student Loan'
 'Personal Loan, Debt Consolidation Loan, and Auto Loan'
 'Not Specified, and Payday Loan'
 'Credit-Builder Loan, Personal Loan, and Auto Loan'
 'Payday Loan, and Payday Loan'
 'Not Specified, Student Loan, and Personal Loan'
 'Personal Loan, Payday Loan, Student Loan, Auto Loan, Home Equity Loan, Student
 Loan, and Payday Loan'
 'Not Specified, Student Loan, Student Loan, Credit-Builder Loan, and Auto Loan'
 'Payday Loan, and Home Equity Loan'
 'Credit-Builder Loan, Not Specified, Mortgage Loan, Payday Loan, Credit-Builder
 Loan, and Personal Loan'
 'Mortgage Loan, Debt Consolidation Loan, Payday Loan, Auto Loan, and Not
 Specified'
 'Credit-Builder Loan, Mortgage Loan, Mortgage Loan, Credit-Builder Loan, and
 Student Loan'
 'Not Specified, Student Loan, and Student Loan'
 'Payday Loan, Not Specified, Credit-Builder Loan, Debt Consolidation Loan,
 Payday Loan, Not Specified, Student Loan, and Student Loan']
```

--- Delay_from_due_date ---

Unique count: 63

```
[ 3  5  8  7 13 10  0  1  9 12 30 31 34 27 14 11  4  2 16 17]
```

--- Num_of_Delayed_Payment ---

```

Unique count: 34
[ 7.  4.  8.  6.  1.  3.  0.  5.  9. 15. 17. 10.  2. 14. 11. 20. 22. 13.
 16. 12.]

--- Changed_Credit_Limit ---
Unique count: 267
[11.27  6.27  9.27  5.42  7.42  6.42  7.1  11.1  9.1  1.99 -2.01 -1.01
 -3.01  2.58 10.14  9.14  9.34  8.34 17.13 21.13]

--- Num_Credit_Inquiries ---
Unique count: 27
[4.000e+00 2.000e+00 3.000e+00      nan 5.000e+00 9.000e+00 8.000e+00
 7.000e+00 6.000e+00 0.000e+00 1.000e+00 1.000e+01 1.050e+03 1.100e+01
 1.200e+01 1.044e+03 1.700e+01 1.936e+03 1.300e+01 5.680e+02]

--- Credit_Mix ---
Unique count: 4
['_' 'Good' 'Standard' 'Bad']

--- Outstanding_Debt ---
Unique count: 125
[ 809.98  605.03 1303.01  632.46  943.86  548.2  352.16 1704.18 1377.74
  421.43 1328.93  950.36  179.22 2602.69  758.44  818.22 1296.64 1283.37
 107.41 2430.21]

--- Credit_Utilization_Ratio ---
Unique count: 834
[26.82261962 31.37786187 27.26225871 22.53759303 23.9337948  24.46403064
 38.55084843 39.18265566 34.97789475 33.3810102  31.13170161 32.93385629
 28.61673482 41.70257342 26.51981539 39.50164811 31.37614957 39.78399346
 38.06862363 38.37475295]

--- Credit_History_Age ---
Unique count: 334
[265. 268. 270. 271.  nan 319. 320. 322. 323. 324. 325. 326. 213. 214.
 215. 217. 218. 219. 220. 207.]

--- Payment_of_Min_Amount ---
Unique count: 3
['No' 'NM' 'Yes']

--- Total_EMI_per_month ---
Unique count: 149
[4.95749492e+01 1.88162146e+01 2.46992319e+02 1.64154517e+01
 0.00000000e+00 1.50150000e+04 1.55150000e+04 1.37644605e+02
 9.11220179e+02 2.38340000e+04 3.26620000e+04 7.04783327e+01
 2.26892792e+02 4.66161291e+01 1.64150000e+04 6.50081743e+01
 1.35173371e+02 8.03570000e+04 1.24392082e+02 3.65481972e+01]

```

--- Amount_invested_monthly ---

Unique count: 772

```
[ 80.41529544 199.45807439 62.43017233 178.34406741
 24.78521651 104.29182517 40.39123783 99.30622796
130.11542024 43.47719014 70.10177421 218.90434353
168.41370268 232.86038376 10000. 825.21626994
430.94752788 257.80809943 263.17416316 81.22885871]
```

--- Payment_Behaviour ---

Unique count: 7

```
['High_spent_Small_value_payments' 'Low_spent_Small_value_payments'
 '!@9#%8' 'High_spent_Medium_value_payments'
 'High_spent_Large_value_payments' 'Low_spent_Medium_value_payments'
 'Low_spent_Large_value_payments']
```

--- Monthly_Balance ---

Unique count: 819

```
[ 312.49408868 223.45130973 340.47921179 244.56531671 358.12416761
 470.69062693 484.59121427 465.67622413 444.86703185 481.50526195
 464.88067789 356.07810856 1043.31597787 998.86929679 715.7413674
 426.51341061 810.78215267 963.92158112 968.55551738 895.49458318]
```

--- Credit_Score ---

Unique count: 3

```
['Good' 'Standard' 'Poor']
```

```
[131]: # Handle special missing value tokens in categorical columns
missing_tokens = ["_", "_____", "!@9#%8", "NM"]

for col in ["Occupation", "Credit_Mix", "Payment_Behaviour",
            ↪ "Payment_of_Min_Amount"]:
    df_clean[col] = df_clean[col].replace(missing_tokens, np.nan)
```

```
[132]: # Re-run the EDA visualizations on clean_df as needed
# Missing values per column in clean_df
missing_counts = df_clean.isnull().sum().sort_values(ascending=False)

# Columns to exclude
exclude_cols = ["SSN"]

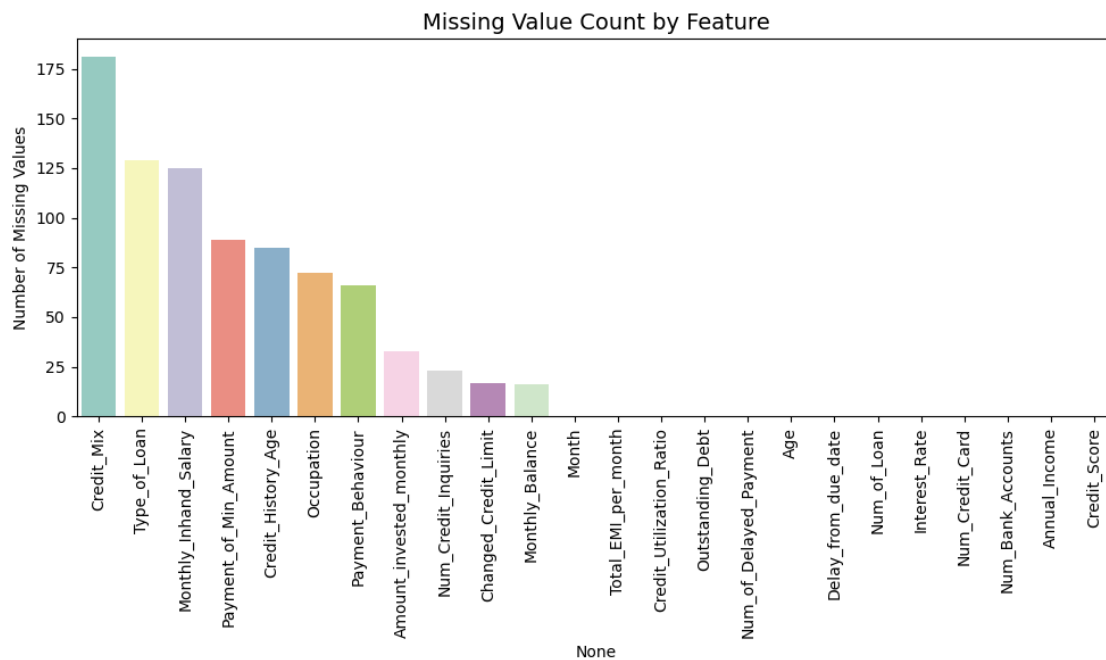
# Remove excluded columns
missing_counts = missing_counts.drop(labels=[c for c in exclude_cols if c in
            ↪ missing_counts.index])

plt.figure(figsize=(10, 6))
sns.barplot(
```

```

x=missing_counts.index,
y=missing_counts.values,
hue=missing_counts.index,
palette="Set3"
)
plt.title("Missing Value Count by Feature", fontsize=14)
plt.ylabel("Number of Missing Values")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

```



```

[133]: # Re-run some of the previous EDA plots on clean_df if needed
# The distribution of Credit_Score after cleaning
plt.figure(figsize=(6,4))
sns.countplot(data=df_clean, x="Credit_Score", hue="Credit_Score",
              palette="Set3")
plt.title("Distribution of Credit Score Classes")
plt.xlabel("Credit Score")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

# Heatmap of correlations in clean_df
numeric_cols = [
    "Age", "Annual_Income", "Monthly_Inhand_Salary", "Num_Bank_Accounts",

```

```

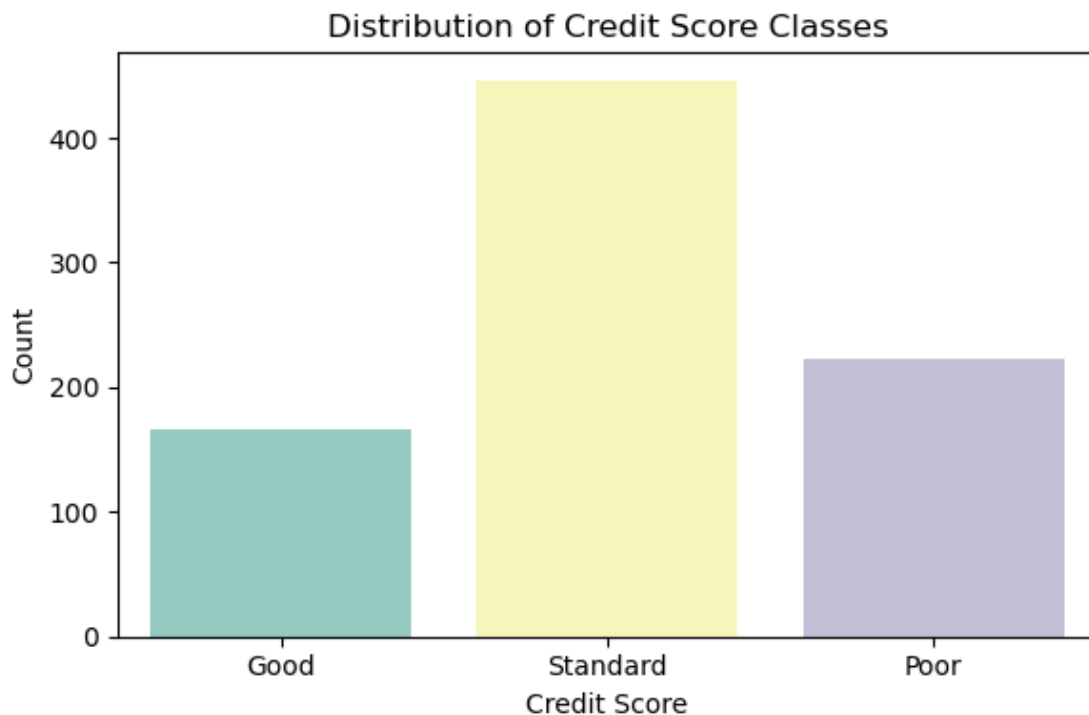
    "Num_Credit_Card", "Interest_Rate", "Num_of_Loan", "Delay_from_due_date",
    "Num_of_Delayed_Payment", "Num_Credit_Inquiries", "Outstanding_Debt",
    "Credit_Utilization_Ratio", "Total_EMI_per_month",
    "Amount_invested_monthly", "Monthly_Balance"
]

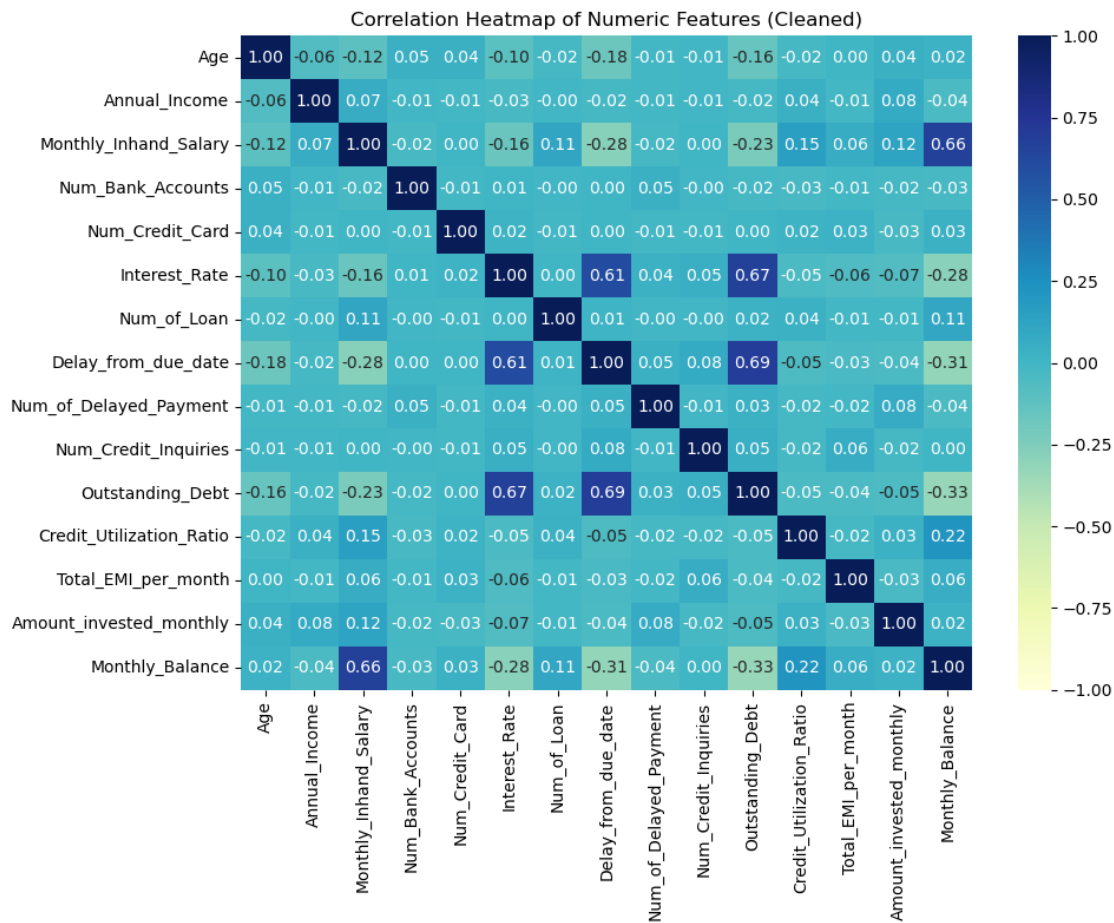
corr = df_clean[numeric_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(
    corr,
    cmap="YlGnBu",
    vmin=-1, vmax=1,
    annot=True, fmt=".2f"
)
plt.title("Correlation Heatmap of Numeric Features (Cleaned)")
plt.tight_layout()
plt.show()

print(df_clean.info())
for col in df_clean.columns:
    uniques = df_clean[col].unique()
    print(f"\n--- {col} ---")
    print(f"Unique count: {len(uniques)}")
    # Only print first 20 to avoid flooding
    print(uniques[:20])

```





```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 834 entries, 0 to 999
```

```
Data columns (total 25 columns):
```

#	Column	Non-Null Count	Dtype
0	Month	834 non-null	object
1	Age	834 non-null	int64
2	SSN	834 non-null	object
3	Occupation	762 non-null	object
4	Annual_Income	834 non-null	float64
5	Monthly_Inhand_Salary	709 non-null	float64
6	Num_Bank_Accounts	834 non-null	int64
7	Num_Credit_Card	834 non-null	int64
8	Interest_Rate	834 non-null	int64
9	Num_of_Loan	834 non-null	int64
10	Type_of_Loan	705 non-null	object

```

11 Delay_from_due_date      834 non-null    int64
12 Num_of_Delayed_Payment   834 non-null    float64
13 Changed_Credit_Limit     817 non-null    float64
14 Num_Credit_Inquiries     811 non-null    float64
15 Credit_Mix               653 non-null    object
16 Outstanding_Debt         834 non-null    float64
17 Credit_Utilization_Ratio 834 non-null    float64
18 Credit_History_Age       749 non-null    float64
19 Payment_of_Min_Amount    745 non-null    object
20 Total_EMI_per_month      834 non-null    float64
21 Amount_invested_monthly  801 non-null    float64
22 Payment_Behaviour        768 non-null    object
23 Monthly_Balance          818 non-null    float64
24 Credit_Score             834 non-null    object
dtypes: float64(11), int64(6), object(8)
memory usage: 169.4+ KB
None

--- Month ---
Unique count: 8
['January' 'April' 'June' 'July' 'August' 'February' 'May' 'March']

--- Age ---
Unique count: 39
[23 28 34 54 55 21 31 33 30 24 44 40 41 35 36 39 37 20 45 46]

--- SSN ---
Unique count: 126
['821-00-0265' '#F%$D@*&8' '004-07-5839' '486-85-3974' '072-31-6145'
'615-06-7821' '612-70-8987' '411-51-0676' '500-92-6408' '070-19-1622'
'366-68-1681' '221-30-8554' '342-90-2649' '414-53-2918' '328-33-6328'
'655-05-7666' '965-46-2491' '891-55-9364' '928-91-4452' '084-25-3745']

--- Occupation ---
Unique count: 16
['Scientist' nan 'Teacher' 'Engineer' 'Entrepreneur' 'Developer' 'Lawyer'
'Media_Manager' 'Doctor' 'Journalist' 'Manager' 'Accountant' 'Musician'
'Mechanic' 'Writer' 'Architect']

--- Annual_Income ---
Unique count: 136
[1.9114120e+04 3.4847840e+04 1.4316264e+05 3.0689890e+04 3.5547710e+04
7.3928460e+04 1.3131340e+05 1.0909427e+07 3.4081380e+04 1.1483841e+05
3.1370800e+04 3.3751270e+04 8.8640240e+04 5.4392160e+04 8.7015450e+03
2.5546260e+04 3.1993780e+04 9.2047080e+04 3.2284620e+04 9.7791420e+04]

--- Monthly_Inhand_Salary ---
Unique count: 134

```

```
[ 1824.84333333      nan  3037.98666667 12187.22
   2612.49083333  2853.30916667  5988.705      11242.78333333
  10469.20775939  2611.115      9843.8675      2825.23333333
   2948.60583333  7266.68666667  4766.68      519.12875
   2415.855      2942.14833333  7591.59      2898.385      ]
```

--- Num_Bank_Accounts ---

Unique count: 16

```
[ 3  2  1  7  4  0  8  5  6  9  10 1414  67  572
  91 528]
```

--- Num_Credit_Card ---

Unique count: 29

```
[ 4  5 1288  1  7  6 1029  488  8 1381  898  3  518 1005
  9 1189  2  10  514  943]
```

--- Interest_Rate ---

Unique count: 31

```
[ 3  6  8  4  5 15  7 12 20  1 14 32 16 17 10 31 25 18 19  9]
```

--- Num_of_Loan ---

Unique count: 12

```
[ 4  1  3  967  0  2  7  5  6  8  9 1464]
```

--- Type_of_Loan ---

Unique count: 98

```
['Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan'
 'Credit-Builder Loan' 'Auto Loan, Auto Loan, and Not Specified'
 'Not Specified' nan 'Credit-Builder Loan, and Mortgage Loan'
 'Not Specified, Auto Loan, and Student Loan'
 'Personal Loan, Debt Consolidation Loan, and Auto Loan'
 'Not Specified, and Payday Loan'
 'Credit-Builder Loan, Personal Loan, and Auto Loan'
 'Payday Loan, and Payday Loan'
 'Not Specified, Student Loan, and Personal Loan'
 'Personal Loan, Payday Loan, Student Loan, Auto Loan, Home Equity Loan, Student
 Loan, and Payday Loan'
 'Not Specified, Student Loan, Student Loan, Credit-Builder Loan, and Auto Loan'
 'Payday Loan, and Home Equity Loan'
 'Credit-Builder Loan, Not Specified, Mortgage Loan, Payday Loan, Credit-Builder
 Loan, and Personal Loan'
 'Mortgage Loan, Debt Consolidation Loan, Payday Loan, Auto Loan, and Not
 Specified'
 'Credit-Builder Loan, Mortgage Loan, Mortgage Loan, Credit-Builder Loan, and
 Student Loan'
 'Not Specified, Student Loan, and Student Loan'
 'Payday Loan, Not Specified, Credit-Builder Loan, Debt Consolidation Loan,
 Payday Loan, Not Specified, Student Loan, and Student Loan']
```

--- Delay_from_due_date ---

Unique count: 63

[3 5 8 7 13 10 0 1 9 12 30 31 34 27 14 11 4 2 16 17]

--- Num_of_Delayed_Payment ---

Unique count: 34

[7. 4. 8. 6. 1. 3. 0. 5. 9. 15. 17. 10. 2. 14. 11. 20. 22. 13.
16. 12.]

--- Changed_Credit_Limit ---

Unique count: 267

[11.27 6.27 9.27 5.42 7.42 6.42 7.1 11.1 9.1 1.99 -2.01 -1.01
-3.01 2.58 10.14 9.14 9.34 8.34 17.13 21.13]

--- Num_Credit_Inquiries ---

Unique count: 27

[4.000e+00 2.000e+00 3.000e+00 nan 5.000e+00 9.000e+00 8.000e+00
7.000e+00 6.000e+00 0.000e+00 1.000e+00 1.000e+01 1.050e+03 1.100e+01
1.200e+01 1.044e+03 1.700e+01 1.936e+03 1.300e+01 5.680e+02]

--- Credit_Mix ---

Unique count: 4

[nan 'Good' 'Standard' 'Bad']

--- Outstanding_Debt ---

Unique count: 125

[809.98 605.03 1303.01 632.46 943.86 548.2 352.16 1704.18 1377.74
421.43 1328.93 950.36 179.22 2602.69 758.44 818.22 1296.64 1283.37
107.41 2430.21]

--- Credit_Utilization_Ratio ---

Unique count: 834

[26.82261962 31.37786187 27.26225871 22.53759303 23.9337948 24.46403064
38.55084843 39.18265566 34.97789475 33.3810102 31.13170161 32.93385629
28.61673482 41.70257342 26.51981539 39.50164811 31.37614957 39.78399346
38.06862363 38.37475295]

--- Credit_History_Age ---

Unique count: 334

[265. 268. 270. 271. nan 319. 320. 322. 323. 324. 325. 326. 213. 214.
215. 217. 218. 219. 220. 207.]

--- Payment_of_Min_Amount ---

Unique count: 3

['No' nan 'Yes']

--- Total_EMI_per_month ---

Unique count: 149

```
[4.95749492e+01 1.88162146e+01 2.46992319e+02 1.64154517e+01
 0.00000000e+00 1.50150000e+04 1.55150000e+04 1.37644605e+02
 9.11220179e+02 2.38340000e+04 3.26620000e+04 7.04783327e+01
 2.26892792e+02 4.66161291e+01 1.64150000e+04 6.50081743e+01
 1.35173371e+02 8.03570000e+04 1.24392082e+02 3.65481972e+01]
```

--- Amount_invested_monthly ---

Unique count: 772

```
[ 80.41529544 199.45807439 62.43017233 178.34406741
 24.78521651 104.29182517 40.39123783 99.30622796
 130.11542024 43.47719014 70.10177421 218.90434353
 168.41370268 232.86038376 10000. 825.21626994
 430.94752788 257.80809943 263.17416316 81.22885871]
```

--- Payment_Behaviour ---

Unique count: 7

```
['High_spent_Small_value_payments' 'Low_spent_Small_value_payments' nan
 'High_spent_Medium_value_payments' 'High_spent_Large_value_payments'
 'Low_spent_Medium_value_payments' 'Low_spent_Large_value_payments']
```

--- Monthly_Balance ---

Unique count: 819

```
[ 312.49408868 223.45130973 340.47921179 244.56531671 358.12416761
 470.69062693 484.59121427 465.67622413 444.86703185 481.50526195
 464.88067789 356.07810856 1043.31597787 998.86929679 715.7413674
 426.51341061 810.78215267 963.92158112 968.55551738 895.49458318]
```

--- Credit_Score ---

Unique count: 3

```
['Good' 'Standard' 'Poor']
```

```
[134]: # Type of loan is nested categorical, so we will manually encode it
# Fill missing with "Missing"
df_clean["Type_of_Loan"] = df_clean["Type_of_Loan"].fillna("Missing")

# Extract unique loan types
def split_loan(text):
    text = str(text).replace("and", "").replace(" ", " ")
    parts = [p.strip() for p in text.split(",") if p.strip() != ""]
    return parts if parts else ["Missing"]

unique_loans = set()
for row in df_clean["Type_of_Loan"]:
    for item in split_loan(row):
        unique_loans.add(item)
```

```

unique_loans = sorted(list(unique_loans))
print("Loan types found:", unique_loans)

# Create binary columns for each loan type and drop original column
for loan in unique_loans:
    df_clean[f"Loan_{loan}"] = df_clean["Type_of_Loan"].apply(
        lambda x: 1 if loan in split_loan(x) else 0
    )
df_clean = df_clean.drop(columns=["Type_of_Loan"])

# Data Splitting and Preprocessing
X = df_clean.drop(columns="Credit_Score")
y = df_clean["Credit_Score"]
groups = df_clean["SSN"]

# GroupShuffleSplit
from sklearn.model_selection import GroupShuffleSplit

gss = GroupShuffleSplit(n_splits=1, test_size=0.15, random_state=42)
train_val_idx, test_idx = next(gss.split(X, y, groups))

X_train_val = X.iloc[train_val_idx].copy()
X_test = X.iloc[test_idx].copy()
y_train_val = y.iloc[train_val_idx].copy()
y_test = y.iloc[test_idx].copy()

X_test = X_test.drop(columns=["SSN"])

# Store the test set for final evaluation later
test_set = X_test.copy()
test_set["Credit_Score"] = y_test.values

test_set.to_csv("../data/test.csv", index=False)
print("Saved to data/test.csv")

```

Loan types found: ['Auto Loan', 'Credit-Builder Loan', 'Debt Consolidation Loan', 'Home Equity Loan', 'Missing', 'Mortgage Loan', 'Not Specified', 'Payday Loan', 'Personal Loan', 'Student Loan']
 Saved to data/test.csv

```

[135]: # Preprocessing pipeline (Handle missing values, categorical encoding, scaling,
      ↪ etc.)
      # Build the column transformer
      from sklearn.experimental import enable_iterative_imputer

      from sklearn.compose import ColumnTransformer
      from sklearn.pipeline import Pipeline

```

```

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder
from sklearn.impute import IterativeImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Identify ordinal and categorical columns

ordinal_cols = ["Credit_Mix", "Payment_of_Min_Amount"]
Credit_Mix_order = ["Missing", "Bad", "Standard", "Good"]
Payment_of_Min_Amount_order = ["Missing", "Yes", "No"]

categorical_cols = ["Month", "Occupation", "Payment_Behaviour"]

numeric_cols = ["Age", "Annual_Income", "Monthly_Inhand_Salary",
↳ "Num_Bank_Accounts", "Num_Credit_Card", "Interest_Rate", "Num_of_Loan",
↳ "Delay_from_due_date", "Num_of_Delayed_Payment", "Changed_Credit_Limit",
↳ "Num_Credit_Inquiries", "Outstanding_Debt",
↳ "Credit_Utilization_Ratio", "Credit_History_Age", "Total_EMI_per_month",
↳ "Amount_invested_monthly", "Monthly_Balance"]

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

ordinal_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("encoder", OrdinalEncoder(categories=[Credit_Mix_order,
↳ Payment_of_Min_Amount_order]))
])

numeric_pipeline = Pipeline([
    ("passthrough", "passthrough")
])

preprocess = ColumnTransformer(
    transformers=[
        ("cat", categorical_pipeline, categorical_cols),
        ("ord", ordinal_pipeline, ordinal_cols),
        ("num", numeric_pipeline, numeric_cols)
    ]
)

full_preprocess = Pipeline([
    ("prep", preprocess),
    ("impute", IterativeImputer(random_state=i)),
    ("scale", StandardScaler())
])

```

```

])

# Define a K-fold cross-validation scheme
from sklearn.model_selection import GroupKFold

groups = X_train_val["SSN"]
X_train_val = X_train_val.drop(columns=["SSN"])

gkf = GroupKFold(n_splits=5)

```

```

[136]: # Hyperparameter tuning for Logistic Regression
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

logreg_scores = []

# Put model in a pipeline with preprocessing
for i in range(5):
    print(f"Round {i+1}: ")
    logreg_pipeline = Pipeline([
        ("prep", full_preprocess),
        ("model", LogisticRegression(
            solver="saga",
            penalty="l2",
            max_iter=8000,
            n_jobs=-1
        ))
    ])

    # Define hyperparameter grid
    param_grid_logreg = {
        "model__C": [0.01, 0.1, 1, 10, 50, 100],
    }

    # Set up GridSearchCV
    logreg_grid = GridSearchCV(
        estimator=logreg_pipeline,
        param_grid=param_grid_logreg,
        cv=gkf,
        scoring="f1_macro",
        n_jobs=-1
    )

    logreg_grid.fit(X_train_val, y_train_val, groups=groups)

    logreg_scores.append(logreg_grid.best_score_)

```

```
print("Best params:", logreg_grid.best_params_)
print("Best CV score:", logreg_grid.best_score_)
```

```
Round 1:
Best params: {'model__C': 0.1}
Best CV score: 0.565426824513238
Round 2:
Best params: {'model__C': 0.1}
Best CV score: 0.565426824513238
Round 3:
Best params: {'model__C': 0.1}
Best CV score: 0.565426824513238
Round 4:
Best params: {'model__C': 0.1}
Best CV score: 0.565426824513238
Round 5:
Best params: {'model__C': 0.1}
Best CV score: 0.565426824513238
```

[137]: *#Hyperparameter tuning for Linear SVM (LinearSVC)*

```
from sklearn.linear_model import SGDClassifier

linear_svc_scores = []

for i in range(5):
    print(f"\nRound {i+1}: ")

    svc_pipeline = Pipeline([
        ("prep", full_preprocess),
        ("model", SGDClassifier(
            loss="hinge",
            penalty="l2",
            max_iter=2000,
            n_jobs=-1,
            tol=1e-3
        ))
    ])

    param_grid_svc = {
        "model__alpha": [1e-4, 1e-3, 1e-2, 1e-1]
    }

    svc_grid = GridSearchCV(
        estimator=svc_pipeline,
        param_grid=param_grid_svc,
        cv=gkf,
```

```

        scoring="f1_macro",
        n_jobs=-1
    )

    svc_grid.fit(X_train_val, y_train_val, groups=groups)

    linear_svc_scores.append(svc_grid.best_score_)

    print("Best params:", svc_grid.best_params_)
    print("Best CV score:", svc_grid.best_score_)

```

Round 1:
 Best params: {'model__alpha': 0.01}
 Best CV score: 0.5483795001714058

Round 2:
 Best params: {'model__alpha': 0.1}
 Best CV score: 0.5510806256139926

Round 3:
 Best params: {'model__alpha': 0.1}
 Best CV score: 0.5543713597647218

Round 4:
 Best params: {'model__alpha': 0.1}
 Best CV score: 0.54303305515844

Round 5:
 Best params: {'model__alpha': 0.1}
 Best CV score: 0.5510543954726604

```

[138]: from sklearn.svm import SVC
        from sklearn.model_selection import GridSearchCV
        from sklearn.pipeline import Pipeline

        svc_rbf_scores = []

        for i in range(5):
            print(f"\nRound {i+1}:")

            # Build pipeline
            svc_pipeline = Pipeline([
                ("prep", full_preprocess),    # preprocessing block
                ("model", SVC(
                    kernel="rbf",
                    probability=False,    # faster

```

```

    ))
])

# Hyperparameter grid
param_grid_svc = {
    "model__C": [0.1, 1, 10],
    "model__gamma": ["scale", "auto"],
}

# GridSearchCV with GroupKFold
svc_grid = GridSearchCV(
    estimator=svc_pipeline,
    param_grid=param_grid_svc,
    cv=gkf,
    scoring="f1_macro",
    n_jobs=-1
)

svc_grid.fit(X_train_val, y_train_val, groups=groups)

svc_rbf_scores.append(svc_grid.best_score_)

print("Best params:", svc_grid.best_params_)
print("Best CV score:", svc_grid.best_score_)

```

Round 1:

Best params: {'model__C': 1, 'model__gamma': 'scale'}

Best CV score: 0.5555472487196401

Round 2:

Best params: {'model__C': 1, 'model__gamma': 'scale'}

Best CV score: 0.5555472487196401

Round 3:

Best params: {'model__C': 1, 'model__gamma': 'scale'}

Best CV score: 0.5555472487196401

Round 4:

Best params: {'model__C': 1, 'model__gamma': 'scale'}

Best CV score: 0.5555472487196401

Round 5:

Best params: {'model__C': 1, 'model__gamma': 'scale'}

Best CV score: 0.5555472487196401

```
[139]: # Hyperparameter tuning for RBF SVM (SVC) NON-LINEAR
from sklearn.svm import SVC

svc_rbf_scores = []

for i in range(5):
    print(f"\nRound {i+1}:")

    # Build pipeline
    svc_pipeline = Pipeline([
        ("prep", full_preprocess),
        ("model", SVC(
            kernel="rbf",
            probability=False,
        ))
    ])

    # Hyperparameter grid
    param_grid_svc = {
        "model__C": [0.01, 0.1, 1, 10, 100],
        "model__gamma": [0.0001, 0.001, 0.01, 0.1, 1],
    }

    # GridSearchCV with GroupKFold
    svc_grid = GridSearchCV(
        estimator=svc_pipeline,
        param_grid=param_grid_svc,
        cv=gkf,
        scoring="f1_macro",
        n_jobs=-1
    )

    svc_grid.fit(X_train_val, y_train_val, groups=groups)

    svc_rbf_scores.append(svc_grid.best_score_)

    print("Best params:", svc_grid.best_params_)
    print("Best CV score:", svc_grid.best_score_)
```

Round 1:

Best params: {'model__C': 100, 'model__gamma': 0.0001}

Best CV score: 0.5676073209903748

Round 2:

Best params: {'model__C': 100, 'model__gamma': 0.0001}

Best CV score: 0.5676073209903748

Round 3:

Best params: {'model__C': 100, 'model__gamma': 0.0001}

Best CV score: 0.5676073209903748

Round 4:

Best params: {'model__C': 100, 'model__gamma': 0.0001}

Best CV score: 0.5676073209903748

Round 5:

Best params: {'model__C': 100, 'model__gamma': 0.0001}

Best CV score: 0.5676073209903748

```
[140]: # Hyperparameter tuning for KNN
from sklearn.neighbors import KNeighborsClassifier

knn_scores = []

for i in range(5):
    print(f"\nRound {i+1}:")

    # Build pipeline
    knn_pipeline = Pipeline([
        ("prep", full_preprocess),      # NaN + scaling pipeline
        ("model", KNeighborsClassifier())
    ])

    # Hyperparameter grid ( + )
    param_grid_knn = {
        "model__n_neighbors": [3, 5, 7, 9, 11],
        "model__weights": ["uniform", "distance"],
        "model__p": [1, 2], # p=1: Manhattan, p=2: Euclidean
    }

    # GridSearchCV with GroupKFold
    knn_grid = GridSearchCV(
        estimator=knn_pipeline,
        param_grid=param_grid_knn,
        cv=gkf,
        scoring="f1_macro",
        n_jobs=-1
    )

    knn_grid.fit(X_train_val, y_train_val, groups=groups)

    knn_scores.append(knn_grid.best_score_)

    print("Best params:", knn_grid.best_params_)
```



```

        handle_unknown="use_encoded_value",
        unknown_value=np.nan,          # unseen categories → NaN
        encoded_missing_value=np.nan # existing NaN → keep NaN
    ), categorical_cols),

    # ordinal: encode with your real order, NaN preserved
    ("ord", OrdinalEncoder(
        categories=[Credit_Mix_order, Payment_of_Min_Amount_order],
        handle_unknown="use_encoded_value",
        unknown_value=np.nan,
        encoded_missing_value=np.nan
    ), ordinal_cols),

    ("num", "passthrough", numeric_cols),      # numeric ok
    ("loan", "passthrough",
        [col for col in X_train_val.columns if col.startswith("Loan_")]),
    ↪ # Loan_xxx passthrough
    ],
    remainder="drop"
)

import xgboost as xgb
from sklearn.pipeline import Pipeline

pipeline_xgboost = Pipeline([
    ("prep", xgb_preprocess),
    ("model", xgb.XGBClassifier(
        objective="multi:softprob",
        eval_metric="mlogloss",
        n_estimators=300,
        random_state=0,
        tree_method="exact"
    ))
])

param_grid_xgb = {
    "model__max_depth": [3, 4, 5],
    "model__learning_rate": [0.05, 0.1],
    "model__subsample": [0.8, 1.0],
    "model__colsample_bytree": [0.8, 1.0],
}

xgb_grid = GridSearchCV(
    estimator=pipeline_xgboost,
    param_grid=param_grid_xgb,
    cv=gkf,
    scoring="f1_macro",

```

```

        n_jobs=-1
    )
    xgb_grid.fit(X_train_val, y_train_val_enc, groups=groups)

    print("Best params:", xgb_grid.best_params_)
    print("Best CV score:", xgb_grid.best_score_)

```

Best params: {'model__colsample_bytree': 1.0, 'model__learning_rate': 0.05, 'model__max_depth': 5, 'model__subsample': 1.0}

Best CV score: 0.6462718378990473

```

[142]: # Hyperparameter tuning for Random Forest
from sklearn.ensemble import RandomForestClassifier

rf_scores = []

for i in range(5):
    print(f"\nRound {i+1}:")

    # Build pipeline
    rf_pipeline = Pipeline([
        ("prep", full_preprocess),
        ("model", RandomForestClassifier(
            random_state=42
        ))
    ])

    # Hyperparameter grid
    param_grid_rf = {
        "model__n_estimators": [200, 400],
        "model__max_depth": [10, None],
        "model__min_samples_split": [2, 5],
        "model__min_samples_leaf": [1, 2],
        "model__max_features": ["sqrt"],
    }

    # GridSearchCV with GroupKFold
    rf_grid = GridSearchCV(
        estimator=rf_pipeline,
        param_grid=param_grid_rf,
        cv=gkf,
        scoring="f1_macro",
        n_jobs=-1
    )

    rf_grid.fit(X_train_val, y_train_val, groups=groups)

```

```
rf_scores.append(rf_grid.best_score_)

print("Best params:", rf_grid.best_params_)
print("Best CV score:", rf_grid.best_score_)
```

Round 1:

Best params: {'model__max_depth': 10, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 2, 'model__min_samples_split': 2,
'model__n_estimators': 400}
Best CV score: 0.6658290958977373

Round 2:

Best params: {'model__max_depth': 10, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 2, 'model__min_samples_split': 2,
'model__n_estimators': 400}
Best CV score: 0.6658290958977373

Round 3:

Best params: {'model__max_depth': 10, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 2, 'model__min_samples_split': 2,
'model__n_estimators': 400}
Best CV score: 0.6658290958977373

Round 4:

Best params: {'model__max_depth': 10, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 2, 'model__min_samples_split': 2,
'model__n_estimators': 400}
Best CV score: 0.6658290958977373

Round 5:

Best params: {'model__max_depth': 10, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 2, 'model__min_samples_split': 2,
'model__n_estimators': 400}
Best CV score: 0.6658290958977373